LuciOS — Software Requirements Specification & Architecture

Core Professional Portfolio, Release 1.1 · Planning Draft · October 22, 2026
Prepared for: Luci Crow

> **Normative for MVP:** The overhead conveyor replaces radial app navigation. The single framed display replaces the planned draggable multi-window manager. Real project screenshot/code galleries replace dummy app previews. Future mystery/evilLuciOS features are out of scope.

1. Purpose, scope and non-goals

LuciOS is a simulated browser-based operating-system interface used to present Luci's real technical projects, academic background, tools and professional contact information. It should be memorable to visitors but straightforward to review for employment or a course grade.

In scope: short simulated boot / Enter experience, rotating overhead rail of hanging app tags, responsive mechanical loader and single app display frame, accessible conventional Portfolio View, About, Projects, Skills, Resume, Contact and local simulated Terminal, three evidence-based project case studies, repository links, source docs, deployment and testing.

Out of scope for v1.1: radial dial, free-floating multi-window manager, draggable/resizable app windows, dummy replicas of project apps, live shell commands, user authentication, production database, real customer ticket data, secret puzzles, garden, page peel, evilLuciOS, completion dialogue. The last four remain future vision only.

2. Stakeholders and usage principles

User	Primary goal
First-time recruiter	Identify Luci, inspect work, download resume, find contact quickly
Technical reviewer	View real screenshots and code choices, follow the repository for each project
Instructor	Verify required portfolio sections, project evidence and reflection
Curious visitor	Enjoy the physical/mechanical app-selection experience
Luci / maintainer	Edit project data, images and URLs without rewriting OS components

Usability rule: The main portfolio never depends on animation, secrets, timed responses, a mouse, or a specific browser feature. A visible direct Portfolio View and Resume/Contact links provide an immediate shortcut.

3. Functional requirements (normative)

Priority: **Must** = MVP release gate; **Should** = desired, may be simplified to meet deadline. Acceptance text is testable.

ID	Priority	Requirement / observable acceptance
FR-01	Must	Site shows a brief boot screen with LuciOS progress treatment and accessible red Enter action; no credentials; skip available.
FR-02	Must	Desktop shell shows an overhead, hanging-app **conveyor rail** with named pixel-style tags for About, Projects, Skills, Resume, Contact and Terminal. No radial app dial.

ID	Priority	Requirement / observable acceptance
FR-03	Must	Visitors can move through the rail from obvious left/right edge controls (pointer hover and keyboard-accessible buttons), select tags by click/Enter, and identify the active selection.
FR-04	Must	Selecting an app displays it within a single, clearly labeled **black display frame** over the desktop; Close/Back restores navigation. Content opens even when motion is disabled.
FR-05	Should	The staged loader depicts app swing, mechanical hand removal/insertion, lowering framed screen, bronze pipes/top-right pipe, gear motion and a brief load effect. Effects must be skippable, safe and cancellable.
FR-06	Must	About provides name, academic program/ interest, career goals and a verified ~100–250 word professional biography.
FR-07	Must	Projects contains **exactly three initial featured entries**: Battleship, CorvusDesk, Bad Apple Repair. Each includes purpose, Luci's contributions, tech stack, decisions, outcome/ lesson, **three real, approved screenshots**, code highlights with explanations and a **verified direct per-project GitHub repo link at the bottom**. Missing evidence must show a clearly labeled pending state during development and must not be presented as complete in final submission.
FR-08	Must	Selecting screenshot/code media opens a larger readable view or accessible equivalent; captions and source labels distinguish application screenshots from code excerpts.
FR-09	Must	Skills presents accurate technical competencies with no unsupported skill ratings.
FR-10	Must	Resume provides a working public-safe PDF download/view action; professional profile links are available when verified.
FR-11	Must	Contact provides a safe professional address or actionable contact link. No private home address or credentials.
FR-12	Must	A conventional **Portfolio View** exposes all core content without using the app rail/loader, and remains reachable from the entry or desktop. Resume and Contact have prominent direct actions.
FR-13	Should	Terminal supports fixed local commands `help`, `about`, `projects`, `skills`, `resume`, `contact`; invalid command shakes text once and shows an error; valid command animates transfer → processing steam → reveal. Steam occurs **only during valid processing after Enter**, not typing or rejection.
FR-14	Must	A separate 100–250 word reflection addresses platform choice, representative project, desired improvements, and career support.

ID	Priority	Requirement / observable acceptance
FR-15	Should	Local non-sensitive preferences (such as reduced motion) may persist; the site still works if persistence is unavailable.

4. Nonfunctional requirements

ID	Category	Target / test
NFR-01	Accessibility	Semantic headings, labeled controls, visible focus, keyboard rail navigation, usable fallback, meaningful alt text; aim for WCAG 2.2 AA principles.
NFR-02	Motion	Honor `prefers-reduced-motion`; show selected content without forced hand motion, gears, steam, wallpaper movement or shakes.
NFR-03	Responsiveness	Desktop visuals first; a basic readable single-column Portfolio View works at 320px+, with no essential horizontal overflow. Full phone-styled LuciOS is postponed.
NFR-04	Performance	Optimize screenshots and fonts, lazy-load lower-priority media, avoid long fake waits; measure startup on target desktop browser.
NFR-05	Security	Terminal is allowlist-only, no arbitrary shell/eval/API execution; escape untrusted content, avoid leaked tokens and customer records.
NFR-06	Privacy	Use only authorized public imagery/code, verified professional contacts and public repository links; minimize data collection.
NFR-07	Reliability	App selection never strands visitors; animation failure still opens content; broken links and assets provide useful fallbacks.
NFR-08	Maintainability	TypeScript models separate records/media from app chrome; use stable IDs and reusable components; project data easily updated.
NFR-09	Compatibility	Verify current Chromium-based browser plus available Firefox/Safari, pointer and keyboard paths; validate deployed HTTPS and resume download.

5. Proposed technical architecture

Stack: React + TypeScript + Vite; CSS variables and SVG animation assets; reducer/context for deterministic UI state; static content assets; hosted as static HTTPS. No custom backend or database required for v1.1.

5.1 Logical route

```
Visitor → Boot + Enter → Desktop Shell
                         ├── OverheadAppRail (scroll/select tags)
                         │       ↓
                         │   App Registry → MechanicalLoader → DisplayFrame → Selected App
                         │                                          └── Typed Content / Media / Verified Repo Link
                         └── Direct Portfolio View ──────────────────────────────┘

Terminal input → Local allowlist validator → [transfer] → [processing valve/steam] → [output]
                         └── invalid → text shake + readable error (no steam)
```

5.2 Component boundaries

Component	Responsibility	Non-responsibility
`BootSequence` / `EnterScreen`	Brief, skippable boot and red 3D entry	Authentication
`DesktopShell`	Wallpaper, rail area, direct shortcuts, accessibility	Project-specific content
`OverheadAppRail`	Ordered tag list, edge scroll, focus/select	Rendering case studies
`AppRegistry`	Stable identifiers, labels, icons, content component mapping	Duplicating content
`MechanicalLoader`	Manage visual phases and cancellations	Controlling actual app data
`DisplayFrame`	One active holographic presentation surface, Back/Close, pipe chrome	Draggable multi-window logic
`ProjectsGallery`	Case-study navigation, screenshots, code highlights, repo link	Fake demo environments
`PortfolioView`	Simple accessible alternative for all content	Hiding sections behind animation
`TerminalApp`	Local command validation, progress state, text response	Remote execution

5.3 State ownership and contracts

The shell owns `selectedAppId`, `railOffset`, `loaderPhase` and `displayState`. The gallery owns only local media selection/lightbox state. Terminal phases live inside Terminal. Keep content records independent from all animation states. **A new selection cancels unfinished animations** and never discards the intended target app. The default is one framed app, not multiple instances. `DisplayFrame` must provide a semantic heading/label and keyboard-close action.

Proposed loader phases: `idle`, `selected`, `retrieving`, `lowering`, `inserting`, `revealing`, `open`, `closing`. These describe visual stages; `open` must be reachable immediately in reduced-motion mode. Optional steam/gears are effects, never network load truth.

5.4 Project record contract

```
type Project = {
  id: 'battleship' | 'corvusdesk' | 'bad-apple';
  title: string;
  purpose: string;
  contributions: string[];
  technologies: string[];
  decisions: string[];
  outcomeOrLessons: string;
  screenshots: { src: string; alt: string; caption: string }[]; // 3 approved, real images
  codeHighlights: { title: string; language: string; code?: string;
                    imageSrc?: string; explanation: string }[];
  repoUrl: string; // verified direct URL; never guess
};
```

During implementation, missing `repoUrl` or images must block the **public-ready** state of that project card; placeholder visuals can be used in developer previews if distinctly labeled and removed before final submission.

5.5 Backend decision

Serve JavaScript, CSS, PDFs and images from static hosting. No API, user accounts, database, server-side command execution or visitor uploads are needed. Any future contact form service or CMS requires separate approval, abuse protections and a revised security model. `localStorage` may hold optional non-sensitive UI settings only.

6. Navigation, failure modes and fallbacks

- Desktop user clicks/focuses an overhead tag → app frame opens; hover scrolling must also have arrow button equivalents, since hover is not universally available.
- `Close` returns to desktop selection; `Back` from a project case study returns to the three-project gallery; Escape dismisses transient overlays with focus restored.
- Reduced motion, slow hardware or animation failure skips the hand/frame choreography but keeps selected content accessible.
- Narrow viewport renders a simplified accessible Portfolio View until a mobile-OS design is approved.
- Broken screenshots show a caption/fallback; broken repo links are a release blocker; no substitute repos may be invented.

7. Evidence and delivery checklist

Deliverable	Evidence / pass gate
Three project entries	Battleship, CorvusDesk, Bad Apple, each with required fields and **three approved app screenshots**
Code highlights	Actual selected code per project with explanations and safe formatting
GitHub	Specific, tested public repo URL **under each project**
Professional	Bio, Skills, Resume, Contact, optional LinkedIn as applicable
Navigation	Rail, direct shortcuts, app view and conventional Portfolio View function
UI behavior	Short boot, red Enter, mechanical framing; no blocking animation
Terminal	Local allowlist; exact steam timing; understandable error on typo
Deployment	Live HTTPS URL and download, basic cross-browser/keyboard tests
Submission	100–250 word reflection, link and summary

8. Short-deadline implementation phases

M0 Scaffold/tests/content shapes → **M1** Portfolio View & evidence → **M2** overhead rail / shell → **M3** single mechanical frame / loader → **M4** project gallery & verified assets → **M5** terminal → **M6** QA/deploy/reflection. Hand articulation, extra particles, advanced conveyor physics and the later evil OS are nonessential polish.

9. Traceability and governing design

Functional IDs in this document are the source of truth for Codex. The UI/UX specification defines wireframes and interaction states; the implementation plan defines module order. The original broader project proposal is inspirational and may discuss later phases, **but it does not authorize those phases for v1.1**. Superseded radial-dial, multi-window and dummy-preview instructions must not be used.

Definition of done: All essential portfolio requirements and working verified evidence are publicly accessible; the site is functional through normal and accessible paths; no private data is exposed; the repository can be built and the deployed URL tested.